

Looking at data

pandas is how you load a table and find out what is in it. These are the calls this course actually uses.

Load it and look at it

<code>pd.read_csv(path)</code>	Read a table. Works directly on a .csv.gz file, no unzipping needed.
<code>df.shape</code>	(rows, columns). The first thing to check, every time.
<code>df.info()</code>	Column names, how many values are present in each, and their types.
<code>df.head(10)</code>	The first ten rows. Useful for orientation, useless for conclusions.
<code>df.describe()</code>	Count, mean, min, max and quartiles for the numeric columns.

Pick out the part you want

<code>df['price']</code>	One column, as a Series.
<code>df[['price', 'district']]</code>	Several columns, as a smaller DataFrame. Note the double brackets.
<code>df.loc[rows, 'price']</code>	Select by label, and the only safe way to assign into a subset.
<code>df.iloc[0:5, 0:3]</code>	Select by position, like a NumPy slice.
<code>df[df['beds'] > 2]</code>	Keep the rows where the condition is true. This is a boolean mask.

Missing values

<code>df.isna().sum()</code>	How many values are missing, per column. Run this before anything else.
<code>df['col'].dropna()</code>	Drop the missing ones. A decision, not a cleanup: see the rule below.
<code>df['col'].fillna(value)</code>	Replace them. Also a decision, and it changes your results.

THE HABIT WORTH FORMING

Look at what you are about to delete, before you delete it.

Rows with a missing value are rarely a random sample of your rows. Compare them with the rows you are keeping first:

`df[df['col'].isna()].describe()` next to `df[df['col'].notna()].describe()`.

If the two look different, dropping changes who your model is about, and you now have something to write down and defend.

Count and summarise

<code>df['district'].value_counts()</code>	How many rows per category, biggest first. Add <code>dropna=False</code> to see the missing ones too.
<code>df.groupby('district')['price'].median()</code>	One number per group. Swap median for mean, count, min, max or std.
<code>df['col'].nunique()</code>	How many distinct values. High for identifiers, low for categories.
<code>df.sort_values('price', ascending=False)</code>	Sort. Handy for finding the extreme rows worth inspecting.

Arrays and figures

NumPy is the layer underneath pandas, and the one Session 8 needs directly. matplotlib is how you ask a question of a picture.

NumPy: it is all about shape

<code>a.shape</code>	The size of each dimension. Print it whenever anything surprises you.
<code>a.sum(axis=0)</code>	axis names the dimension that disappears. axis=0 collapses the rows.
<code>a.reshape(3, -1)</code>	Rearrange. -1 means work the remaining dimension out for me.
<code>a * b</code>	Element-wise, position by position. Shapes must match or broadcast.
<code>a @ b</code>	Matrix product. This is the one a network layer needs.
<code>a[a > 0]</code>	Boolean mask: keep the elements where the condition holds.

SHAPE DISCIPLINE

Write the shape in a comment on the line that creates the array.

NumPy will happily broadcast two shapes you did not intend into a result that is the wrong answer and raises no error at all.

When something is wrong, print shapes before you print values. Most array bugs are visible in one line of shapes.

Figures: use the object API

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(df['price'], bins=50)
ax.set_xlabel('price per night')
ax.set_ylabel('listings')
ax.set_title('What does the price distribution look like?')
```

fig is the canvas, ax is the panel you draw on. This is the form that composes into multi-panel figures later.

<code>ax.hist(x, bins=50)</code>	One variable: what is the shape of this distribution?
<code>ax.scatter(x, y, alpha=0.3)</code>	Two numeric variables: do they move together? alpha helps with over-plotting.
<code>ax.boxplot</code> / <code>sns.boxplot</code>	A number split by a category: does the level differ between groups?
<code>sns.histplot(df, x='p', hue='g')</code>	seaborn takes the DataFrame directly. Pass ax=ax to place it.

BEFORE YOU DRAW ANYTHING

You do not choose a chart. You choose a question, and the question chooses the chart.

Write the question in the title. If you cannot state the question a figure answers, you have decoration, not evidence.

Four ways a chart misleads: a truncated axis, an average that hides the shape, over-plotting that hides the density, and a third variable explaining the pattern you think you see.